Category:

Reversing

Name:

Gorilla is not Godzilla

Message:

You are given a log file of received messages from Gorillabot C2, and a defanged Gorillabot binary for analysis. Please analyze them. **Your PC would detect the binary as malware even it is defanged. You must prepare an isolated environment for malware analysis.**

Extraction password is "virus".

Warning: **DO NOT EXECUTE THE BINARY CODE IN YOUR IMPORTANT ENVIRONMENT.** We defanged the binary in a minimal way so that players do not execute its malicious code unintentionally. If you break the guard, we consider you accept the risk of executing malware code that could damage your system. We highly recommend to use a disassembler or a decompiler for analysis and read the resulted code carefully, not a debugger until you prepare an isolated virtual machine for malware analysis. **Please understand that we are not responsible for any damage resulting from code execution of the binary. We confirmed this challenge could be solved without code execution.**

To proof your understanding, please create flag by "CSG_FLAG{<$NTH_IP_IN _DISCORD_ATTACKS$>}", where <$NTH_IP_IN_DISCORD_ATTACKS$> is the n-th largest IP address of the targets in DISCORD attacks from the message log. And submit it to the system.
Example format: CSG_FLAG{127.0.0.1}

Note: Maybe you know IP addresses are comparable in their integer representation. For example, "8.8.8.8" = 0x08080808 = 134744072 < "127.0.0.1" = 0x7F000001 = 2130706433.

Instructions:

You can see message.log contains time and binary.
Your task is to decrypt the binary to know what attack message is given from the C2 server in that time.

There are articles analyzing gorillabot:

- https://nsfocusglobal.com/over-300000-gorillabot-the-new-king-of-ddos-attacks/

- https://medium.com/@anyrun/gorillabot-technical-analysis-and-code-similarities-with-mirai-cf8d40386870

Read the articles carefully, you can find the gorillabot uses a custom XTEA-like cipher for secure C2 communication. The decryption algorithm is explained there.

So you can copy the code to decode them. You need to take care of byte-level operations if you try implement the code in Python.

SBOX

```
SBOX=binascii.unhexlify('''
 63 7C 77 7B F2 6B 6F C5
 30 01 67 2B FE D7 AB 76  CA 82 C9 7D FA 59 47 F0
 AD D4 A2 AF 9C A4 72 C0  B7 FD 93 26 36 3F F7 CC
 34 A5 E5 F1 71 D8 31 15  04 C7 23 C3 18 96 05 9A
 07 12 80 E2 EB 27 B2 75  09 83 2C 1A 1B 6E 5A A0
 52 3B D6 B3 29 E3 2F 84  53 D1 00 ED 20 FC B1 5B
 6A CB BE 39 4A 4C 58 CF  D0 EF AA FB 43 4D 33 85
 45 F9 02 7F 50 3C 9F A8  51 A3 40 8F 92 9D 38 F5
 BC B6 DA 21 10 FF F3 D2  CD 0C 13 EC 5F 97 44 17
 C4 A7 7E 3D 64 5D 19 73  60 81 4F DC 22 2A 90 88
 46 EE B8 14 DE 5E 0B DB  E0 32 3A 0A 49 06 24 5C
 C2 D3 AC 62 91 95 E4 79  E7 C8 37 6D 8D D5 4E A9
 6C 56 F4 EA 65 7A AE 08  BA 78 25 2E 1C A6 B4 C6
 E8 DD 74 1F 4B BD 8B 8A  70 3E B5 66 48 03 F6 0E
 61 35 57 B9 86 C1 1D 9E  E1 F8 98 11 69 D9 8E 94
 9B 1E 87 E9 CE 55 28 DF  8C A1 89 0D BF E6 42 68
 41 99 2D 0F B0 54 BB 16  '''.replace('\n','').replace(' ',''))
```

Decrypt all (includes key calculation)

```
def decrypt_buffer(buf, key, subkeys):
    length = len(buf)
    if length <= 7:
        raise Exception(f'[ERROR] decrypt_buffer invalid length {length}')
    i = 0
    dec_buf = bytes()
    while i < length:
        dec = decrypt_feistel(buf[i:i+8], key, subkeys)
        dec_buf += dec
        i = i+8
    return dec_buf

def decrypt(msg):
    length = struct.unpack('>H', msg[0:2])[0]
    if len(msg) != length + 2:
        raise Exception(f'[ERROR] Invalid message! len={length}, actual_len={len(msg)}')
    msg = msg[2:]
    expect_sig = msg[8:40] # sha256 signature
    key0 = struct.unpack('>Q', msg[0:8])[0]
    key_list = []
    for i in range(64):
        idx = 40 + 4*i
        key_list.append(struct.unpack('>I',msg[idx:idx+4])[0])
    enc_msg = msg[296:]
    dec = decrypt_buffer(enc_msg, key0, key_list)
    return dec[:-dec[-1]] # Remove padding
```

Decrypt 8-bytes unit

```
def decrypt_feistel(buf, key, subkeys):
    i = 63
    v4 = struct.unpack('>Q', buf[0:8])[0]
    v5 = (key & 0xFFFFFFFF00000000) >> 32 # HIDWORD
    v6 = key & 0xFFFFFFFF
    v7 = (v4 ^ key) & 0xFFFFFFFF
    v8 = v5 ^ ((v4 & 0xFFFFFFFF00000000) >> 32)

    # print(f'[DEBUG1] {hex(v4)}, {hex(v5)}, {hex(v6)}, {hex(v7)}, {hex(v8)}')
    
    for i in range(63, -1, -1):
        if (i & 1) == 0:
            v9 = subkeys[i] ^ v7
            v11 = v8 ^ 0xCAFEBABE
            # print(f'[DEBUG3]{i}:v9={hex(subkeys[i])} XOR {hex(v7)} = {hex(v9)}, v11_tmp = {hex(v8)} XOR 0xCAFEBABE = {hex(v11)}')
            v10 = ror32(
                SBOX[BYTE0(v9)] |
                (SBOX[BYTE1(v9)] << 8) |
                (SBOX[BYTE2(v9)] << 16) |
                (SBOX[BYTE3(v9)] << 24)
                ,29
            )
            x = BYTE0(BYTE_SUB(v10,3)) | ((BYTE_SUB(BYTE1(v10),3)) << 8) | ((BYTE_SUB(BYTE2(v10),3)) << 16) | ((BYTE_SUB(BYTE3(v10),3)) << 24)
            
            v11 ^= x
            v8 = v7
            if i != 0:
                v7 = v11
            # print(f'[DEBUG2] {i}: {hex(v9)}, {hex(v10)}, {hex(v11)}, {hex(v7)}, {hex(v8)}')
        else: # i%2=1
            v12 = (subkeys[i] + v7) & 0xFFFFFFFF
            v11 = v8 ^ 0xDEADBEEF
            # print(f'[DEBUG3]{i}:v12={hex(subkeys[i])} XOR {hex(v7)} = {hex(v12)}, v11_tmp = {hex(v8)} XOR 0xDEADBEEF = {hex(v11)}')
            
            v13 = ror32(
                SBOX[BYTE0(v12)] |
                (SBOX[BYTE1(v12)] << 8) |
                (SBOX[BYTE2(v12)] << 16) |
                (SBOX[BYTE3(v12)] << 24)
                ,5
            )
            x = BYTE0(BYTE_ADD(v13,5)) | ((BYTE_ADD(BYTE1(v13),5)) << 8) | ((BYTE_ADD(BYTE2(v13),5)) << 16) | ((BYTE_ADD(BYTE3(v13),5)) << 24)
            v11 ^= x
            v8 = v7
            if i != 0:
                v7 = v11
            # print(f'[DEBUG2] {i}: {hex(v12)}, {hex(v13)}, {hex(v11)}, {hex(v7)}, {hex(v8)}')

    # print(f'[DEBUG4] v6={hex(v6)}, v11={hex(v11)}, v7={hex(v7)}, v5={hex(v5)}')
    result = (v6 ^ v11) | ((v7 ^ v5) << 32)
    
    # print(f'[DEBUG]dec_result={hex(result)}')
    return struct.pack('>Q', result)
```

Then you can decrypt the message and get the attack message. According to NSFocus's analysis, the attack number 18 is mapped to "attack_udp_discord", what we want to filter them to get the flag. Checking the target IP addresses of them, you will get the flag.

In this challenge, the flag depends on N.

N	Flag
1	CSG_FLAG{221.158.149.212}
2	CSG_FLAG{218.152.242.38}
3	CSG_FLAG{211.205.207.35}
4	CSG_FLAG{188.53.61.89}
5	CSG_FLAG{182.248.0.34}
6	CSG_FLAG{182.214.214.106}
7	CSG_FLAG{175.197.164.45}
8	CSG_FLAG{175.197.124.124}
9	CSG_FLAG{172.104.62.47}
10	CSG_FLAG{162.159.136.234}
11	CSG_FLAG{162.159.135.234}
12	CSG_FLAG{162.159.135.232}
13	CSG_FLAG{162.159.130.234}
14	CSG_FLAG{162.159.128.235}
15	CSG_FLAG{162.159.61.3}
16	CSG_FLAG{159.0.61.35}
17	CSG_FLAG{146.103.26.136}
18	CSG_FLAG{135.125.166.29}
19	CSG_FLAG{128.116.44.102}
20	CSG_FLAG{128.116.13.33}